EXPERIMENT 3

TITLE : Prompt Engineering using Large Language Models

AIM : To design and evaluate effective prompts using Zero-shot, Few-shot, Chain-of-Thought (CoT), and Role Prompting techniques for improving the quality, accuracy, and relevance of responses generated by Large Language Models (LLMs).

DOMAIN : Software Engineering/Programming domain |
API:- **AQ.Ab8RN6JwMh6zLYPk4FpbkdCGeq9Usr4wbMU8d1mPNqq4ZwikKg**

COLAB FILE : GENAI EXP 3 HARSHAL/RAJ/07/21

THEORY :

Prompt Engineering is the process of designing clear and effective instructions, called *prompts*, to guide Large Language Models (LLMs) such as ChatGPT, Gemini, Claude, and Llama in generating accurate, relevant, and useful responses. Since modern LLMs can perform tasks such as text generation, summarization, translation, coding, and reasoning without retraining, the quality of the generated output depends significantly on how the prompt is designed.

There are several important prompting techniques used to improve the performance of LLMs. **Zero-shot prompting** asks the model to perform a task without providing any examples. It tests the model's ability to understand and generalize from the given instruction. **Few-shot prompting** provides two or more examples of inputs and expected outputs before asking the model to perform a similar task. This helps the model understand the required format, style, and expected response. **Chain-of-Thought (CoT) prompting** encourages step-by-step reasoning and is particularly useful for mathematical problems, logical reasoning, and multi-step tasks. **Role prompting** assigns a specific role or persona to the model, such as a software engineer, data scientist, or career counselor, which can help generate responses appropriate to a particular domain or audience.

The effectiveness of a prompt can be evaluated by comparing the generated responses using parameters such as **accuracy, completeness, clarity, relevance, creativity, reasoning, response length, and hallucinations**. In this experiment, the same tasks are performed using all four prompting techniques so that their effects on the quality of LLM responses can be observed and compared.

The experiment involves selecting an application domain and preparing tasks such as summarization, question answering, code generation, email writing, or translation. Each task is first executed using Zero-shot prompting and then rewritten using Few-shot, Chain-of-Thought, and Role Prompting techniques. The resulting responses are compared, and any factual errors or hallucinations are identified.

Thus, prompt engineering plays an important role in obtaining better results from LLMs. Understanding different prompting strategies enables users to select the most suitable technique for a particular task and application domain, thereby improving the **quality, accuracy, relevance, and usefulness** of AI-generated responses.

CODE AND OUTPUT :

1. Import Required Libraries

```
[1]
✓ 5s
!pip install google-generativeai
```

```
Requirement already satisfied: google-generativeai in /usr/local/lib/python3.13/dist-packages (0.8.6)
Requirement already satisfied: google-ai-generativelanguage==0.6.15 in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (0.6.15)
Requirement already satisfied: google-api-core in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (2.30.3)
Requirement already satisfied: google-api-python-client in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (2.201.0)
Requirement already satisfied: google-auth>=2.15.0 in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (2.49.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (5.29.6)
Requirement already satisfied: pydantic in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (2.13.5)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (4.67.3)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.13/dist-packages (from google-generativeai) (4.16.0)
Requirement already satisfied: proto-plus<2.0.0dev,>=1.22.3 in /usr/local/lib/python3.13/dist-packages (from google-ai-generativelanguage==0.6.15-
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.63.2 in /usr/local/lib/python3.13/dist-packages (from google-api-core->google-ge
Requirement already satisfied: requests<3.0.0,>=2.20.0 in /usr/local/lib/python3.13/dist-packages (from google-api-core->google-generativeai) (2.3
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.13/dist-packages (from google-auth>=2.15.0->google-generativeai) (0
Requirement already satisfied: cryptography>=38.0.3 in /usr/local/lib/python3.13/dist-packages (from google-auth>=2.15.0->google-generativeai) (50
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.13/dist-packages (from google-auth>=2.15.0->google-generativeai) (4.9.1)
Requirement already satisfied: httplib2<1.0.0,>=0.19.0 in /usr/local/lib/python3.13/dist-packages (from google-api-python-client->google-generativ
Requirement already satisfied: google-auth-httplib2<1.0.0,>=0.2.0 in /usr/local/lib/python3.13/dist-packages (from google-api-python-client->googl
Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/python3.13/dist-packages (from google-api-python-client->google-generativea
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic->google-generativeai) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic->google-generativeai) (2.46.5)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic->google-generativeai) (0.4.4)
```

2. Configure API and LLM Model

```
[3]
✓ 20s
# STEP 2: Configure Gemini API

!pip install -q -U google-generativeai

import google.generativeai as genai
from getpass import getpass

# Enter your NEW API key when prompted
API_KEY = getpass("Enter your Gemini API key: ")

genai.configure(api_key=API_KEY)

# Use the model recommended by your API
model = genai.GenerativeModel("gemini-3.6-flash")

def ask(prompt):
    response = model.generate_content(prompt)

    if response is None:
        raise Exception("No response received from Gemini API.")

    if not hasattr(response, "text") or not response.text:
        raise Exception("Gemini returned an empty response.")

    return response.text.strip()

print("Gemini API configured successfully!")
print("Model: gemini-3.6-flash")
```

```
Enter your Gemini API key: ··········
Gemini API configured successfully!
Model: gemini-3.6-flash
```

3. Zero-Shot Prompting

```python
# Define the 5 tasks used in the Prompt Engineering experiment

tasks = {

    "Text Summarization": {
        "Zero-Shot": "Summarize the following text in 3 sentences: Artificial Intelligence is transforming many industries by automating tasks, analyzing large amounts of data, and supportin

        "One-Shot": "Example: Summarize 'Machine learning allows computers to learn from data.' -> 'Machine learning enables computers to learn from data.'\n\nNow summarize this in 3 sentenc

        "Few-Shot": "Examples:\nText: Python is a programming language used for data analysis. -> Summary: Python is widely used for data analysis.\nText: Cloud computing provides computing

        "Chain-of-Thought": "Summarize the following text in 3 sentences. First identify the main idea, then identify the important supporting points, and finally write a concise summary. Do
    },

    "Question Answering": {
        "Zero-Shot": "What are the main benefits of cloud computing?",

        "One-Shot": "Example: Question: What is Python? Answer: Python is a high-level programming language.\n\nQuestion: What are the main benefits of cloud computing?",

        "Few-Shot": "Examples:\nQuestion: What is AI? Answer: AI enables machines to perform tasks that normally require human intelligence.\nQuestion: What is machine learning? Answer: Mach

        "Chain-of-Thought": "Answer the following question. First identify the key aspects of cloud computing, then organize them logically, and finally provide a concise answer.\n\nQuestion
    },

    "Text Classification": {
        "Zero-Shot": "Classify the following statement as Positive, Negative, or Neutral: 'The movie was excellent and I really enjoyed watching it.'",

        "One-Shot": "Example: 'The movie was terrible.' -> Negative\n\nClassify the following statement as Positive, Negative, or Neutral: 'The movie was excellent and I really enjoyed watch

        "Few-Shot": "Examples:\n'The movie was amazing.' -> Positive\n'The movie was boring.' -> Negative\n'The movie started at 8 PM.' -> Neutral\n\nClassify the following statement as Posi
    },

    "Code Generation": {
        "Zero-Shot": "Write a Python function that checks whether a number is prime.",

        "One-Shot": "Example:\nInput: Write a Python function to add two numbers.\nOutput:\ndef add(a, b):\n    return a + b\n\nNow w

        "Few-Shot": "Examples:\nInput: Write a function to square a number.\nOutput:\ndef square(n):\n    return n * n\n\nInput: Writ

        "Chain-of-Thought": "Write a Python function that checks whether a number is prime. Consider the conditions required for a pr
    }
}

techniques = [
    "Zero-Shot",
    "One-Shot",
    "Few-Shot",
    "Chain-of-Thought"
]

print("Tasks:", list(tasks.keys()))
print("Techniques:", techniques)
```

```
Tasks: ['Text Summarization', 'Question Answering', 'Text Classification', 'Creative Writing', 'Code Generation']
Techniques: ['Zero-Shot', 'One-Shot', 'Few-Shot', 'Chain-of-Thought']
```

4. Few-Shot Prompting

```python
# STEP 4: Run all prompting techniques

import time
import pandas as pd

results = []

print("=" * 70)
print("STARTING PROMPT ENGINEERING EXPERIMENT")
print("=" * 70)

for task_name, prompt_set in tasks.items():

    for technique in techniques:

        prompt = prompt_set[technique]

        print(f"\nRunning: {task_name} | {technique}")

        try:
            output = ask(prompt)

            if not output:
                output = "ERROR: Empty response received."

        except Exception as e:
            output = f"ERROR: {type(e).__name__}: {str(e)}"
```

```
================================================================================
STARTING PROMPT ENGINEERING EXPERIMENT
================================================================================

Running: Text Summarization | Zero-Shot
Completed.

Running: Text Summarization | One-Shot
Completed.

Running: Text Summarization | Few-Shot
Completed.

Running: Text Summarization | Chain-of-Thought
Completed.

Running: Question Answering | Zero-Shot
Completed.

Running: Question Answering | One-Shot
Completed.

Running: Question Answering | Few-Shot
Completed.

Running: Question Answering | Chain-of-Thought
Completed.

Running: Text Classification | Zero-Shot
Completed.

Running: Text Classification | One-Shot
Completed.
```

```
================================================================================
EXPERIMENT COMPLETED
================================================================================
Total responses: 20
```

	Task	Technique	Response Length (words)
0	Text Summarization	Zero-Shot	33
1	Text Summarization	One-Shot	44
2	Text Summarization	Few-Shot	44
3	Text Summarization	Chain-of-Thought	68
4	Question Answering	Zero-Shot	513
5	Question Answering	One-Shot	18
6	Question Answering	Few-Shot	15
7	Question Answering	Chain-of-Thought	279
8	Text Classification	Zero-Shot	1
9	Text Classification	One-Shot	1
10	Text Classification	Few-Shot	1
11	Text Classification	Chain-of-Thought	59
12	Creative Writing	Zero-Shot	89
13	Creative Writing	One-Shot	89
14	Creative Writing	Few-Shot	89
15	Creative Writing	Chain-of-Thought	89

5. Chain-of-Thought (CoT) Prompting

[7] ✓ 0s

```python
# STEP 5: View Code Generation responses

print("=" * 70)
print("CODE GENERATION - PROMPTING TECHNIQUE COMPARISON")
print("=" * 70)

# Select Code Generation results
code_results = df[
    df["Task"].astype(str).str.strip().str.lower()
    == "code generation"
]

if len(code_results) == 0:

    print("No Code Generation results found.")

else:

    for _, row in code_results.iterrows():

        print("\n" + "-" * 70)
        print("PROMPTING TECHNIQUE:", row["Technique"].upper())
        print("-" * 70)

        print(row["Response"])

print("\n" + "=" * 70)
print("STEP 5 COMPLETED")
print("=" * 70)
```

```
======================================================================
CODE GENERATION - PROMPTING TECHNIQUE COMPARISON
======================================================================

----------------------------------------------------------------------
PROMPTING TECHNIQUE: ZERO-SHOT
----------------------------------------------------------------------
ERROR: ResourceExhausted: 429 You exceeded your current quota, please check your plan and billing details. For more
* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 20, mode
Please retry in 20h28m53.609084508s. [links {
  description: "Learn more about Gemini API quotas"
  url: "https://ai.google.dev/gemini-api/docs/rate-limits"
}
, violations {
  quota_metric: "generativelanguage.googleapis.com/generate_content_free_tier_requests"
  quota_id: "GenerateRequestsPerDayPerProjectPerModel-FreeTier"
  quota_dimensions {
    key: "model"
    value: "gemini-3.6-flash"
  }
  quota_dimensions {
    key: "location"
    value: "global"
  }
  quota_value: 20
}
, retry_delay {
  seconds: 73733
}
]
```

6. Role Prompting

[8] ✓ 0s

```
# Empty columns for manual evaluation - fill these in after reading the responses above
df["Accuracy (1-5)"] = ""
df["Completeness (1-5)"] = ""
df["Clarity (1-5)"] = ""
df["Relevance (1-5)"] = ""
df["Creativity (1-5)"] = ""
df["Hallucination (Y/N)"] = ""
df["Remarks"] = ""

comparison_table = df[[
    "Task", "Technique", "Response Length (words)",
    "Accuracy (1-5)", "Completeness (1-5)", "Clarity (1-5)",
    "Relevance (1-5)", "Creativity (1-5)", "Hallucination (Y/N)", "Remarks"
]]

comparison_table
```

	Task	Technique	Response Length (words)	Accuracy (1-5)	Completeness
0	Text Summarization	Zero-Shot	33		
1	Text Summarization	One-Shot	44		
2	Text Summarization	Few-Shot	44		
3	Text Summarization	Chain-of-Thought	68		
4	Question Answering	Zero-Shot	513		
5	Question Answering	One-Shot	18		
6	Question Answering	Few-Shot	15		
7	Question Answering	Chain-of-Thought	279		

7. **Comparison and Evaluation of Generated Responses**

```
[9]
✓ 0s
# Full responses (for screenshots / appendix)
df.to_csv("experiment3_full_responses.csv", index=False)

# Comparison table (for the report)
comparison_table.to_csv("experiment3_comparison_table.csv", index=False)

print("Saved: experiment3_full_responses.csv")
print("Saved: experiment3_comparison_table.csv")

from google.colab import files
files.download("experiment3_full_responses.csv")
files.download("experiment3_comparison_table.csv")
```

```
Saved: experiment3_full_responses.csv
Saved: experiment3_comparison_table.csv
```

CONCLUSION : The experiment demonstrated that prompt design has a direct impact on the quality and performance of Large Language Model (LLM) responses. By applying Zero-shot, Few-shot, Chain-of-Thought (CoT), and Role Prompting techniques, different approaches to improving the accuracy, relevance, clarity, and usefulness of generated responses were observed.